



코멘토 실무PT 클래스

반도체 SW 엔지니어를 위한 새로운 SoC 위에 리눅스 디바이스 드라이버 포팅하기

멘티님, 반갑습니다!

클래스는 20:00에 시작됩니다.

출석 확인을 위해 프로필은 **실명**으로 설정해주세요.

출석을 확인합니다.



김대용

김진영

원종진

조현경

최승원

김동현

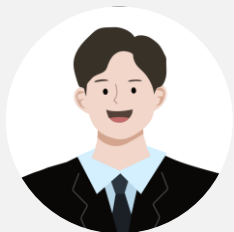
최민우

최강주

라이브로 함께할 서로에 대해 알아보아요.

- 1 멘티님이 하고 있는 일에 대해 소개해주세요.
- 2 이번 실무PT를 신청한 이유를 알려주세요.
- 3 실무PT가 끝난 뒤, 달성하고 싶은 목표를 알려주세요.
- 4 클래스에 참여하며 걱정되는 부분이 있다면 알려주세요.

먼저, 저에 대해 먼저 소개해 드릴게요 😊



뚚뚚개미

학사 : 전기공학 / 컴퓨터공학 복수전공

- 전기 자동차 연구 (1저자, SCI 저널)
- 임베디드 소프트웨어 경진대회 최우수상

석사 : 컴퓨터공학

- 운영체제, 보안 연구 (2저자, SCI 저널)

前 국내 대기업 연구 개발 부문 소프트웨어 엔지니어

- Security Top tier 학회 논문 게재 (1저자)

Ex. 現 해외 기업 소프트웨어 엔지니어

- 차량용 시스템 소프트웨어 개발

이제, 여러분의 이야기를 들려주세요.

- 1 멘티님이 하고 있는 일에 대해 소개해주세요.
- 2 이번 실무PT를 신청한 이유를 알려주세요.
- 3 실무PT가 끝난 뒤, 달성하고 싶은 목표를 알려주세요.
- 4 클래스에 참여하며 걱정되는 부분이 있다면 알려주세요.

우리의 목표를 알려 드릴게요.

1

저는 이렇게 사용하고 있어요.

시스템 반도체 개발할 때
해당 칩 위에서 동작하도록
리눅스 포팅 작업이 필요해요

새로운 하드웨어를 개발하면
어떤 식으로 드라이버를 개발
할 수 있을까요?

2

**이런 부분만큼은
제가 도와드릴 수 있어요.**

비싼 장비 없이도 QEMU를
사용하여 하드웨어의 구조를
하나하나 파헤쳐 볼 수 있어요

현업에서 사용할 수 있는 스킬과
관련된 개념들을 같이 파악하도록
도와드릴게요

3

**모든 과정이 끝나면
여러분도 할 수 있어요.**

리눅스 디바이스 드라이버의
전문가로 가기 위한 첫 걸음을
내딛으실 수 있어요

하드웨어가 어떤 형태로 연결이
되고 상호 작용하는지 개념을
잡아보실 수 있어요

주차 별 커리큘럼을 소개합니다.

1주차 : 새로운 SoC 설계하기 첫걸음

2주차 : MMIO를 사용하는 하드웨어 설계 및 개발

3주차 : DMA를 사용하는 하드웨어 설계 및 개발

4주차 : GPIO를 사용하는 하드웨어 설계 및 개발

5주차 : I2C 버스를 사용하는 하드웨어 설계 및 개발

6주차 : SPI 버스를 사용하는 하드웨어 설계 및 개발

시작하기에 앞서

1

수업 진행을 하며
멘티님들의
이해도를 체크하며
진행할 예정입니다.

2

이해가 되지 않거나,
궁금한 내용은
채팅으로 언제든지
남겨주세요.

3

평균 50분 라이브를
진행하고, 10분의
쉬는 시간으로
진행 됩니다.

01 개발 환경 구축하기

02 QEMU로 새로운 SoC 설계하기

03 CPU 파악하기

04 디바이스 트리 알아보기

05 새 SoC에서 리눅스 부팅하기

Chapter. 1

개발 환경 구축하기

수업에 사용할 개발 환경

본 수업에서는 우분투 22.04 버전을 기본 개발 환경으로 사용합니다.

- 다른 버전의 우분투나 배포판 사용가능 – 단, 이슈 발생시 대처 어려움

우분투 22.04 버전 설치하는 방법

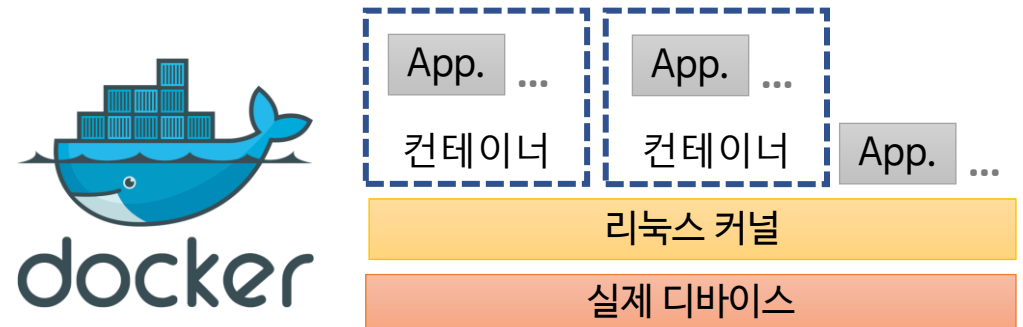
- Virtualbox 등 가상화 프로그램 사용
 - [반도체 SW 엔지니어를 위한 리눅스 디바이스 드라이버 개발하기](#) 클래스 참고
 - 클래스에서 사용했던 가상머신 그대로 사용 가능
- Docker를 사용하여 우분투 이미지를 사용
 - 기존에 사용하는 우분투의 버전이 다를 때
 - WSL(Windows Subsystem for Linux) 을 사용할 때

Docker 란?

<https://www.docker.com/>

리눅스 환경을 컨테이너 기술을 사용하여 이미지로 관리하고 실행하는 도구

- 원하는 버전의 배포판을 쉽게 다운로드 받아 사용 가능
- 작업 환경을 이미지의 형태로 쉽게 공유 가능
- 특정 개발에만 사용되는 복잡한 환경이나 프로그램을 컨테이너 안에서만 유지
 - 특정 개발에만 사용되는 프로그램을 컨테이너 안에서만 설치하게 가능
 - 개발이 끝나면 컨테이너만 삭제하면 되므로 불필요한 프로그램 제거 용이

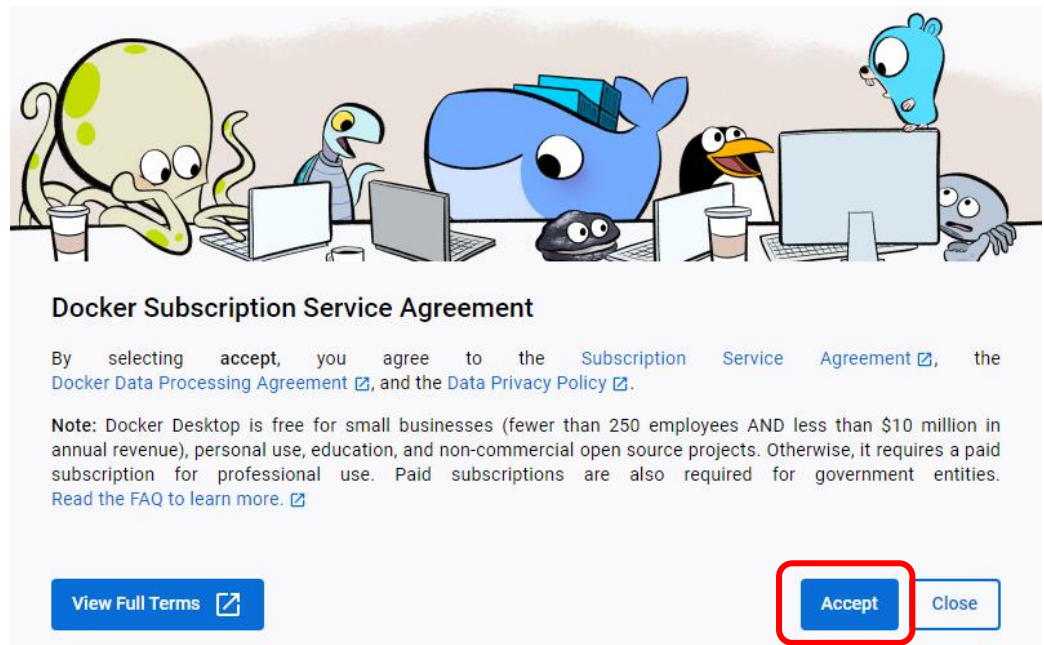
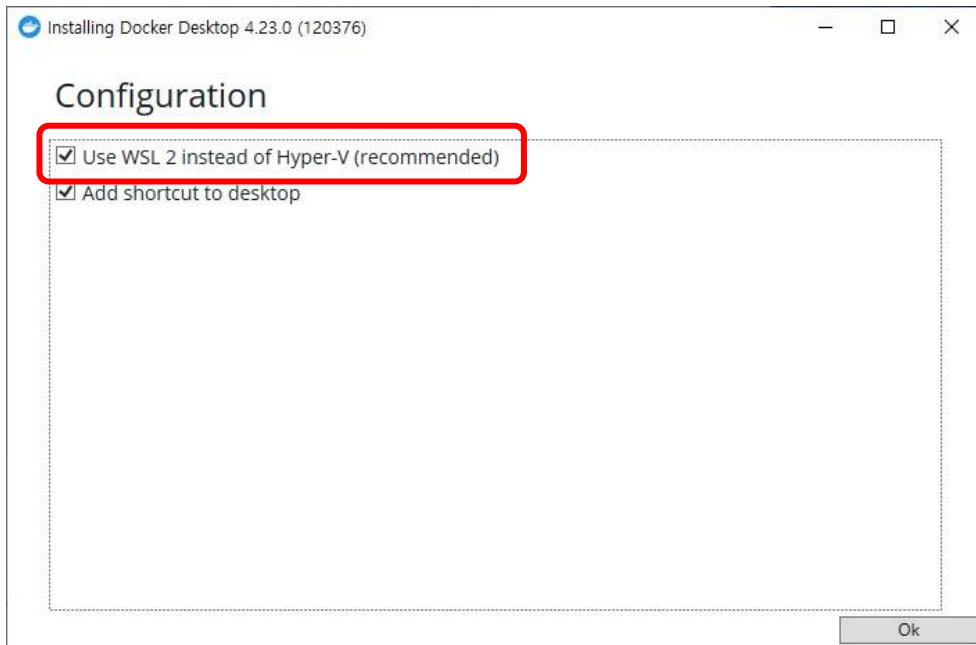


Docker 설치하기

<https://docs.docker.com/get-docker/>

윈도우, 리눅스, MacOS 등 각자 환경에 맞는 Docker desktop 다운로드

- 윈도우의 경우 아래와 같이 설치 (Use WSL 2)
- Docker Subscription Service Agreement는 Agree



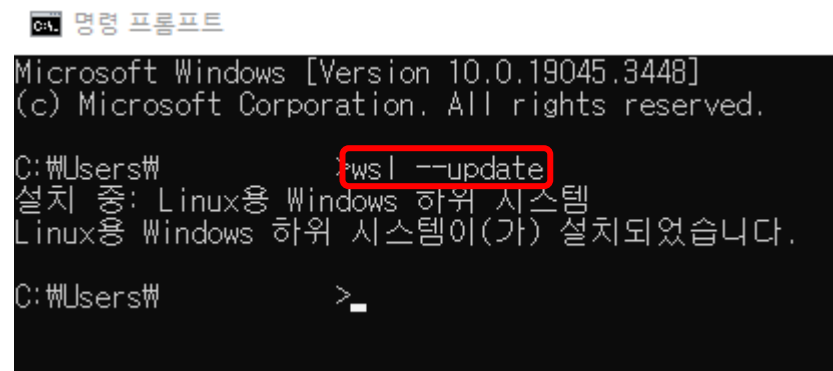
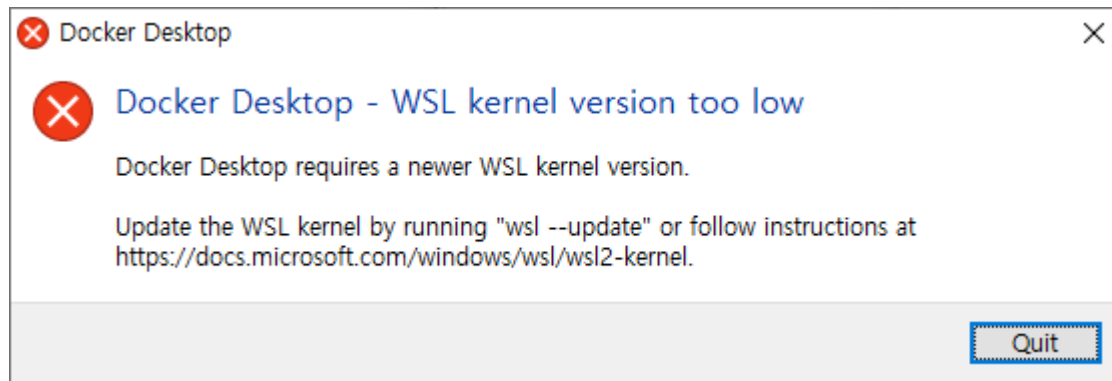
Docker 설치하기 - WSL 버전 업데이트

윈도우에서 Docker 사용을 위해서는 WSL 버전의 업데이트가 필요

- 버전이 낮다면 아래와 같은 메시지가 나옴

윈도우 명령어 프롬프트를 실행하고 wsl --update 명령어 실행

- 윈도우 명령어 프롬프트 : 윈도우키 + R 이후, 실행 창에서 cmd 입력 후 엔터



Docker 실행하기

Docker를 실행하기 위해서는 Docker Desktop 실행 필요

- 이후, 명령 프롬프트에서 Docker 명령어 실행 가능

컨테이너 만들기

- `docker run --privileged -dit -v <공유할 디렉토리>:/workspace --name comento ubuntu:22.04`

컨테이너 접속하기 : `docker attach comento`

컨테이너 다시 시작하기 : `docker start comento`

새로운 셸 더 띄우기 : `docker exec -it comento /bin/bash`

```
C:\Users>docker run -dit --name comento ubuntu:22.04
Unable to find image 'ubuntu:22.04' locally
22.04: Pulling from library/ubuntu
445a6a12be2b: Pull complete
Digest: sha256:aabed3296a3d45ceded1dc866a24476c4d7e093aa806263c27ddaadbdc3c1054
Status: Downloaded newer image for ubuntu:22.04
1b2fc8632d5a0594e7f2e5b59a850c0d9f521b364086b47db71cd843ef9e5a58

C:\Users>docker attach comento
root@1b2fc8632d5a:/# ls
bin boot dev etc home lib lib32 lib64 libx32 media mnt opt proc root
```

커널, 빌드루트, QEMU 다운로드 하기

컨테이너 기본 환경 설정

- apt-get update
- apt-get install sudo wget vim xz-utils

아래와 같은 명령어로 커널, 빌드 루트, QEMU 다운로드

- cd ~
- wget <https://cdn.kernel.org/pub/linux/kernel/v6.x/linux-6.5.5.tar.xz>
- wget <https://buildroot.org/downloads/buildroot-2023.08.tar.gz>
- wget <https://download.qemu.org/qemu-8.0.5.tar.xz>
- tar xvf linux-6.5.5.tar.xz
- tar xvf buildroot-2023.08.tar.gz
- tar xvf qemu-8.0.5.tar.xz

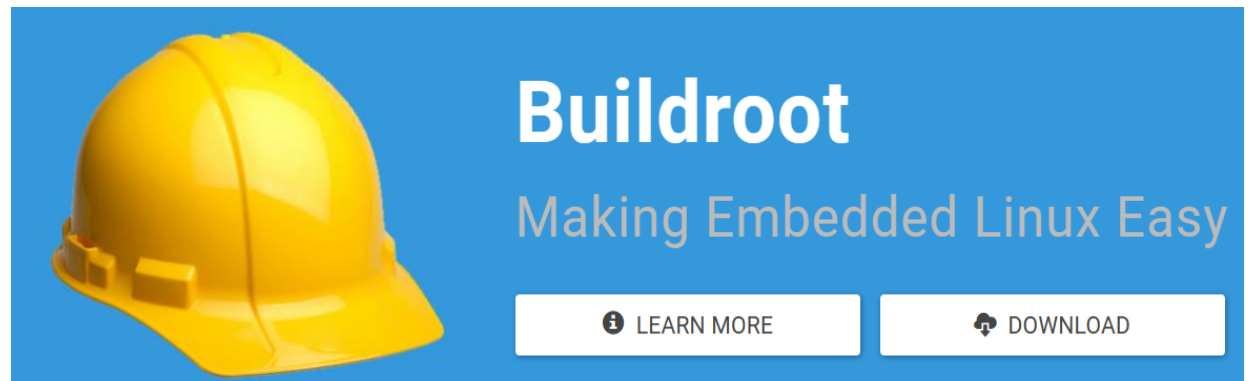
빌드 루트 (Buildroot)

<https://buildroot.org/>

오픈소스 기반의 맞춤형 루트 파일 시스템을 만들어 주는 프로젝트

특징

- 빌드 시스템은 Makefile로 구성되어 있음
- 필요한 요소들은 자동으로 다운로드 하여 빌드함
- 빌드 루트를 사용하지 않는다면, 개발자가 각각의 모든 요소에 대해
 1. 소스 코드를 찾아서 다운로드,
 2. 다운로드 받은 소스코드를 빌드,
 3. 빌드된 바이너리를 이미지로 생성의 과정들을 수동으로 수행하여야 함



빌드 루트 빌드 하기

빌드 의존성 패키지 설치

- `sudo apt install build-essential file bison flex cpio unzip rsync bc libncurses-dev ₩ --no-install-recommends`

빌드 루트 설정하기

- `cd buildroot-2023.08; make qemu_aarch64_virt_defconfig; make menuconfig`
 - Toolchain -> Toolchain type : External toolchain
 - Filesystem images -> cpio the root filesystem : 선택
 - Filesystem images -> cpio the root filesystem -> Compression method : gzip
 - Kernel -> Linux kernel : 선택 안함
 - Host utilities : 모든 항목 선택 안함

빌드 루트 빌드하기

- `make -j<코어 개수>`

커널 빌드 하기

GCC 툴체인 대신 LLVM 사용 가능

- `sudo apt install clang lld llvm --no-install-recommends`

커널 설정하기

- `cd linux-6.5.5`
- `cp ../buildroot-2023.08/board/qemu/aarch64-virt/linux.config arch/arm64/configs/comento_defconfig`
- `ARCH=arm64 LLVM=1 make comento_defconfig`

커널 빌드하기

- `ARCH=arm64 LLVM=1 make -j<코어 개수>`

QEMU

<https://www.qemu.org/>

오픈소스의 다양한 CPU와 디바이스를 지원하는 에뮬레이터
전체 시스템 에뮬레이션, 유저 모드 에뮬레이션, 가상화 지원

- 본 수업에서는 전체 시스템 에뮬레이션을 사용



QEMU 빌드 하기

빌드 의존성 패키지 설치

- `sudo apt install meson ninja-build pkg-config libglib2.0-dev libpixman-1-dev ₩ --no-install-recommends`

QEMU 설정하기

- `cd qemu-8.0.5`
- `mkdir build; cd build; ../configure --target-list="aarch64-softmmu" --without-default-features`

QEMU 빌드하기

- `cd build; make -j<코어 개수>`

빌드 테스트

커널, 빌드루트, QEMU가 모두 잘 빌드 되었는지 확인해보기

- <QEMU 디렉토리>/build/qemu-system-aarch64 ₩
-kernel <리눅스 디렉토리>/arch/arm64/boot/Image ₩
-drive format=raw,file=<빌드루트 디렉토리>/output/images/rootfs.ext4,if=virtio ₩
-append "root=/dev/vda console=ttyAMA0" ₩
-nographic -M virt -cpu cortex-a72 ₩
-m 2G ₩
-smp 2

Welcome to Buildroot 까지 뜨면 성공!

- 로그인 ID : root, 비밀번호 : 없음
- QEMU 종료하기 : poweroff -f



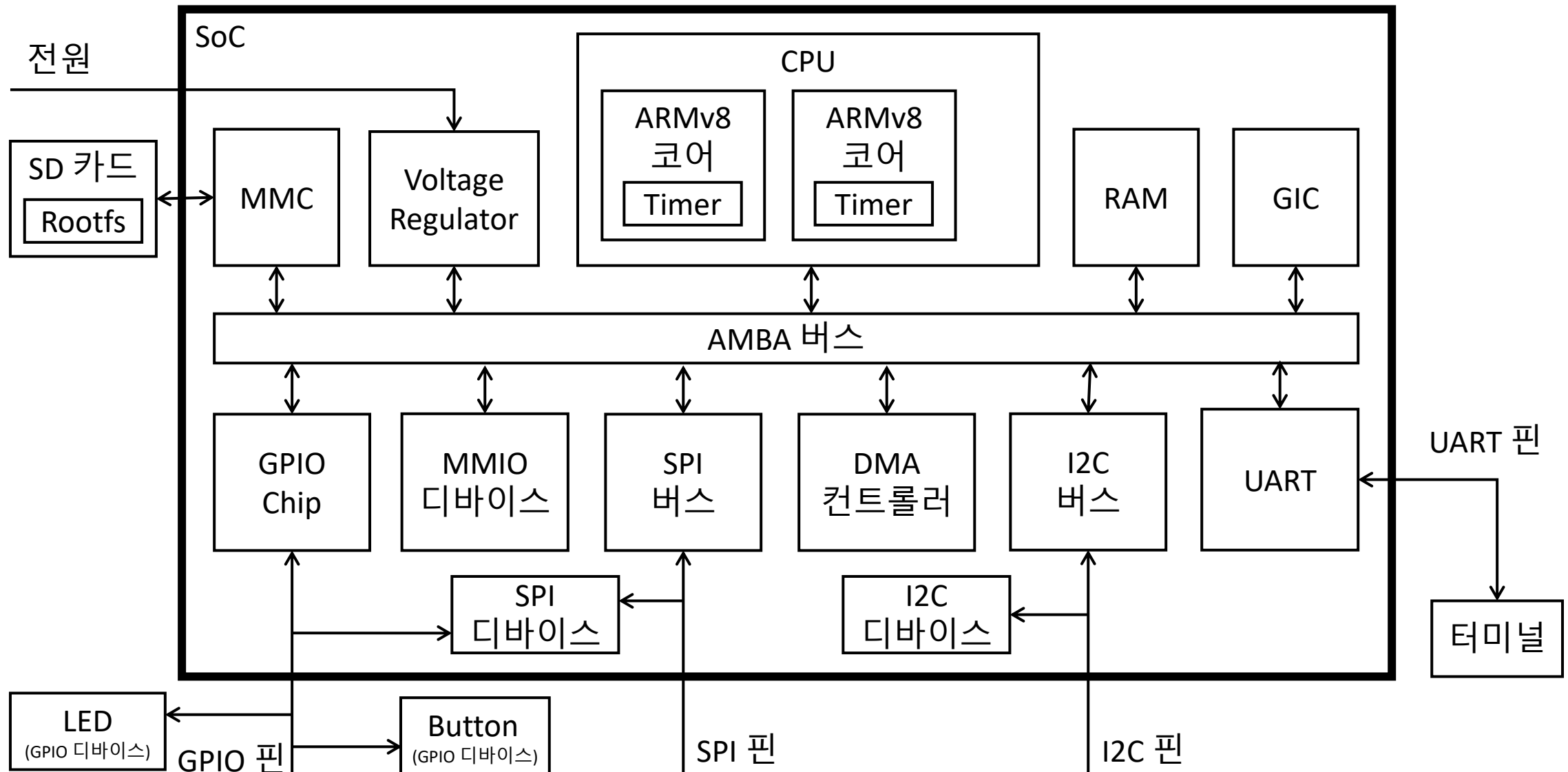
질문해주세요!

이해되지 않거나, 궁금한 내용은 채팅과 마이크를 통해 질문해주세요.
질문이 많을수록 더 많은 걸 알려드릴 수 있어요.

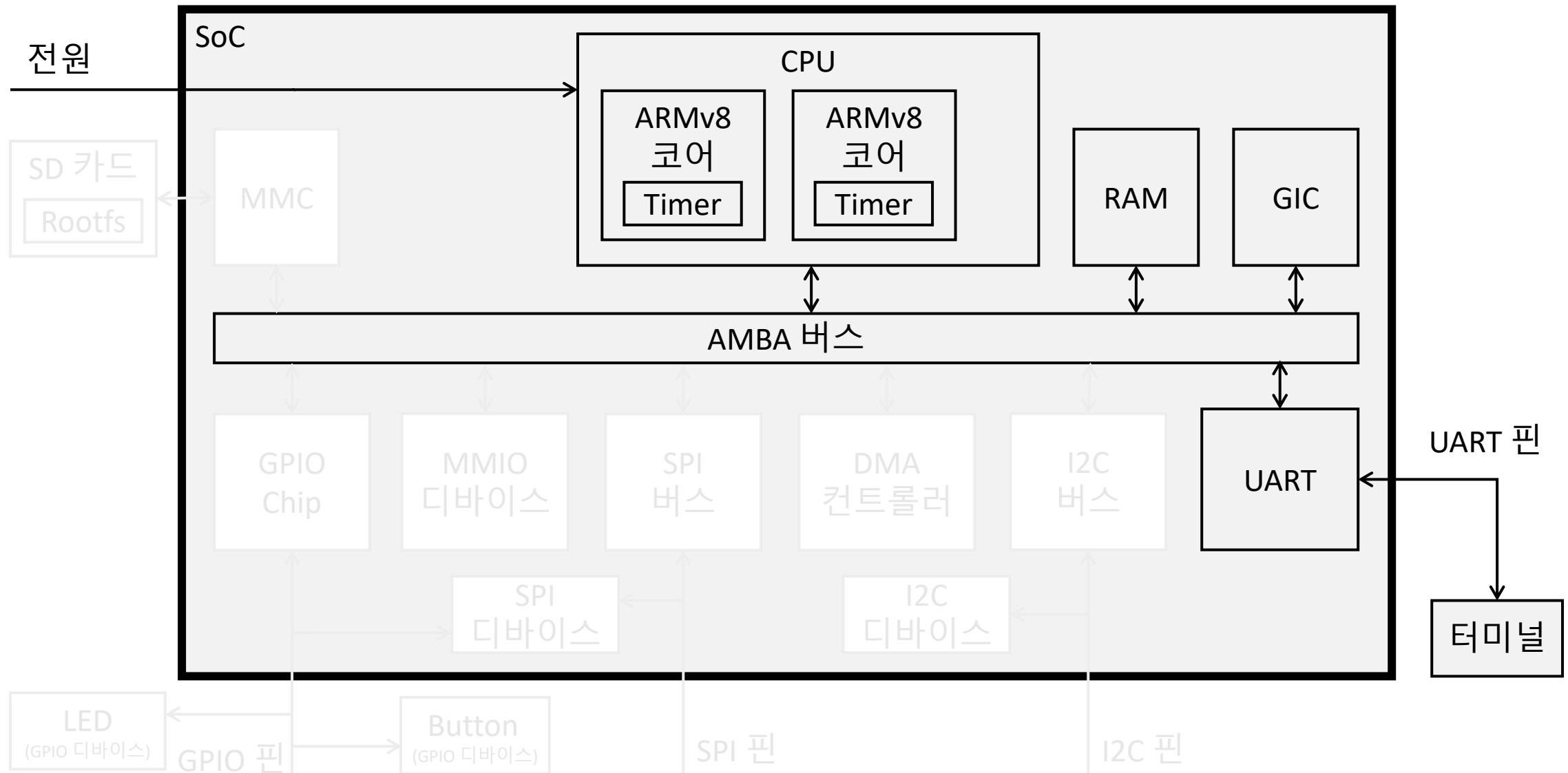
Chapter. 2

QEMU에 새로운 SoC 설계 하기

수업 동안 만들어 나갈 SoC 구조



오늘 수업에서 진행할 내용



QEMU에 새로운 소스코드 추가하기

1. 새로운 config를 아래와 같이 Kconfig 마지막에 추가

```
config COMENTO
    bool
    imply I2C_DEVICES
    select ARM_GIC
    select PL011 # UART
    select PL031 # RTC
    select PL181 # mmc
    select PL330 # dma
    select PL022 # SPI
    select PL061 # GPIO
    select ARM_SBCON_I2C # I2C
```

<hw/arm/Kconfig>

2. 추가한 config가 적용되도록 아래와 같이 default.mak에 추가

```
CONFIG_COMENTO=y <configs/devices/arm-softmmu/default.mak>
```

3. 새로 추가할 소스 코드가 빌드 되도록 meson.build에 추가

```
arm_ss.add(when: 'CONFIG_COMENTO', if_true: files('comento.c'))
```

<hw/arm/meson.build>

QEMU에 새로운 SoC 추가하기

<hw/arm/comento.c>

```
#include "qemu/osdep.h"
#include "qemu/datadir.h"
#include "qemu/units.h"
#include "qemu/option.h"
#include "qemu/module.h"
#include "qemu/error-report.h"
#include "qapi/error.h"
#include "hw/boards.h"
#include "hw/sysbus.h"
#include "hw/loader.h"
#include "hw/qdev-properties.h"
#include "hw/arm/boot.h"
#include "sysemu/runstate.h"
#include "sysemu/sysemu.h"
#include "target/arm/cpu.h"

#define COMENTO_BASE_MEM 0x40000000
#define TYPE_COMENTO_MACHINE MACHINE_TYPE_NAME("comento")

struct ComentoMachineClass {
    MachineClass parent;
};

struct ComentoMachineState {
    MachineState parent;
    struct arm_boot_info bootinfo;
};

OBJECT_DECLARE_TYPE(ComentoMachineState, ComentoMachineClass, \
                    COMENTO_MACHINE)

static void machcomento_init(MachineState *machine)
{
```

```
    ComentoMachineState *vms = COMENTO_MACHINE(machine);
    MemoryRegion *systemem = get_system_memory();
    memory_region_add_subregion(systemem, COMENTO_BASE_MEM, machine->ram);
    vms->bootinfo.ram_size = machine->ram_size;
    vms->bootinfo.loader_start = COMENTO_BASE_MEM;
    arm_load_kernel(ARM_CPU(first_cpu), machine, &vms->bootinfo);
}

static void comento_machine_class_init(ObjectClass *oc, void *data)
{
    MachineClass *mc = MACHINE_CLASS(oc);
    mc->desc = "Comento board";
    mc->init = machcomento_init;
    mc->max_cpus = 128;
    mc->block_default_type = IF_SD;
    mc->no_cdrom = 1;
    mc->minimum_page_bits = 12;
    mc->default_ram_id = "mach-comento.ram";
}

static const TypeInfo comento_machine_info = {
    .name = "comento-machine",
    .parent = TYPE_MACHINE,
    .instance_size = sizeof(ComentoMachineState),
    .class_size = sizeof(ComentoMachineClass),
    .class_init = comento_machine_class_init,
};

static void machcomento_machine_init(void)
{
    type_register_static(&comento_machine_info);
}

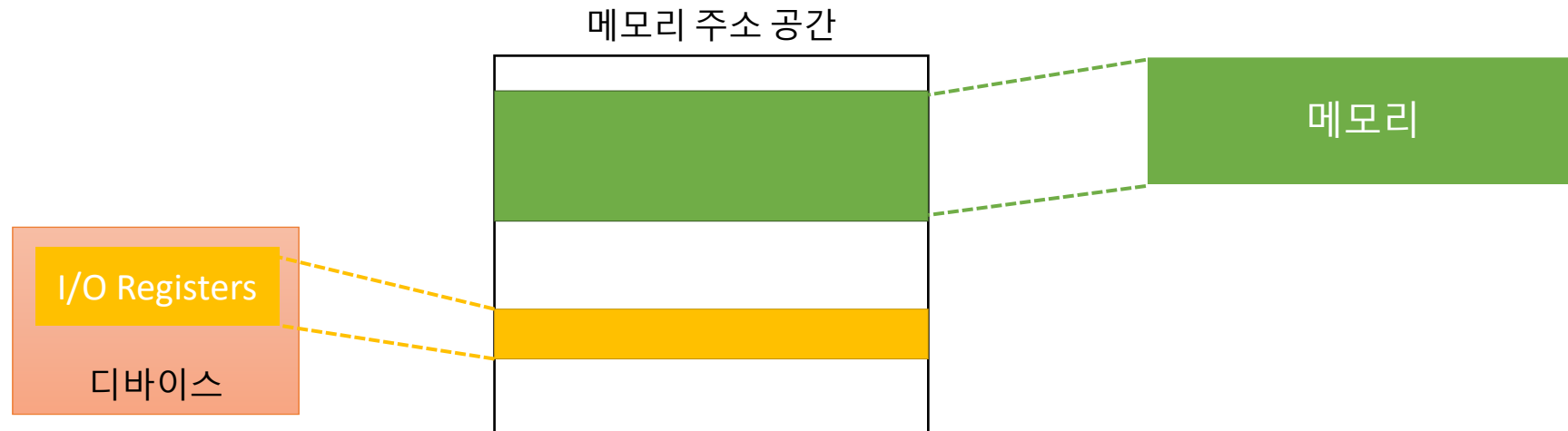
type_init(machcomento_machine_init);
```

메모리 맵 (Memory map)

ARM 64bit CPU에서 사용 가능한 물리 메모리 주소 크기 : 48bit (상위 64~48bit는 다른용도)

실제 메모리나 디바이스의 입출력 레지스터가 메모리 주소 점유

- 실제 메모리가 0 번지 주소부터 시작하지 않아도 됨
- 아무도 점유하고 있지 않는 주소를 사용하면 예외 발생



➤ Memory Mapped I/O 수업에서 좀더 자세히 다루겠습니다.

새로운 SoC 실행하기

QEMU 새로 빌드 하기

- `cd build; make -j<코어 개수>`

새로 추가한 SoC로 부팅하기

- `<QEMU 디렉토리>/build/qemu-system-aarch64 ₩`
`-kernel <리눅스 디렉토리>/arch/arm64/boot/Image ₩`
`-append "console=ttyAMA0" ₩`
`-nographic -M comento ₩`
`-m 1G -smp 2`

아직 Segmentation fault만 뜨고 실행되지는 않음

- SoC에서 제일 중요한 부분인 CPU가 없기 때문



질문해주세요!

이해되지 않거나, 궁금한 내용은 채팅과 마이크를 통해 질문해주세요.
질문이 많을수록 더 많은 걸 알려드릴 수 있어요.



쉬는 시간!

조금 쉬었다가, 다시 만나요!

Chapter. 3

CPU 파악하기

ARM Microarchitecture

ARM사에서 반도체에서 CPU core를 위해 제공하는 IP

- IP란? 칩을 설계하기 위한 Building block을 의미
 - 반도체 제조사는 원하는 IP를 사서 조립하여 하나의 SoC로 제작하게 됨

ARM Cortex 시리즈

- Cortex-A : 스마트폰 등에 사용되는 Application Processor 위한 라인업
 - 리눅스 등 일반 운영체제 설치 가능
- Cortex-R : 실시간성을 하드웨어적으로도 보장하기 위한 라인업
 - Cortex-A와 가장 중요한 차이 : 가상 주소 미 지원, 예외 처리 방식
- Cortex-M : 소형 장치에 사용되는 Microcontroller를 위한 라인업
 - Cortex-R과 유사하나 좀더 저전력과 저가형 기기에 초점

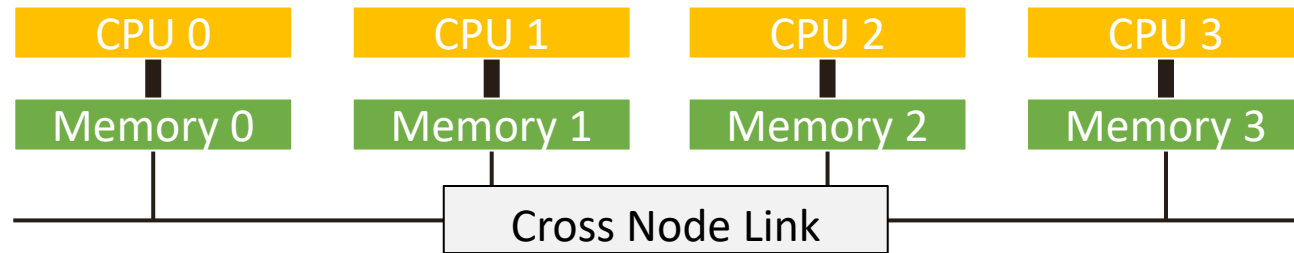
➤ 본 수업에서는 Cortex-A만 다루도록 하겠습니다.

The ARM logo, consisting of the lowercase letters 'arm' in a bold, blue, sans-serif font.

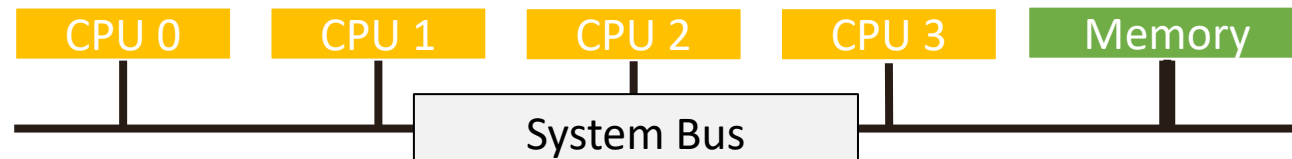
NUMA vs SMP

NUMA(Non-Uniform Memory Access) : CPU 각각에 메모리 할당

- 자신에게 할당된 메모리가 아닌 다른 메모리는 Cross Node Link를 통해 접근 (속도 느림)



SMP (Symmetric Multi-Processor) : CPU가 하나의 메모리를 공유



➤ 본 수업에서는 SMP 구조를 사용합니다.

PSCI (Power State Coordination Interface) 란?

리눅스 부팅시에는 0번 CPU만 켜짐

- 부팅 과정 중에 다른 CPU들의 전원을 켜고 초기화하게 됨

불필요한 전력 소모를 막기 위해 리눅스는 CPU를 켜거나 끌 수 있음

PSCI가 도입되기 전에는 SoC마다 CPU 전원을 관리하는 리눅스 코드가 달랐음

- 실제 구현은 다르더라도 리눅스의 코드는 같게 유지하자!
 - 리눅스는 PSCI를 사용하여 CPU를 켜거나/끄거나/정지하기 위한 요청을 보냄

PSCI는 커널 모드에서 요청하는 것이므로 시스템콜과 비슷한 명령어 필요

- HVC (하이퍼바이저로 요청), SMC (TrustZone의 Monitor 모드로 요청) 사용

QEMU의 새 SoC에 CPU 추가하기

```
...
#define COMENTO_CPU_TYPE ARM_CPU_TYPE_NAME("cortex-a72")
#define COMENTO_BASE_GIC_DIST 0x09000000
...
static void machcomento_init(MachineState *machine)
{
...
    int index;
    unsigned int smp_cpus = machine->smp_cpus;

    for (index = 0; index < smp_cpus; index++) {
        Object *cpuobj;
        CPUState *cs;
        cpuobj = object_new(COMENTO_CPU_TYPE);
        object_property_set_bool(cpuobj, "has_el3", false, NULL);
        object_property_set_bool(cpuobj, "has_el2", false, NULL);
        if (object_property_find(cpuobj, "reset-cbar")) {
            object_property_set_int(cpuobj, "reset-cbar",
                                    COMENTO_BASE_GIC_DIST, &error_abort);
        }
        object_property_set_link(cpuobj, "memory", OBJECT(sysmem),
                                &error_abort);
        qdev_realize(DEVICE(cpuobj), NULL, &error_fatal);
        cs = CPU(cpuobj);
        cs->cpu_index = index;
        object_unref(cpuobj);
    }
    vms->bootinfo.psci_conduit = QEMU_PSCI_CONDUIT_HVC;
...
    arm_load_kernel(ARM_CPU(first_cpu), machine, &vms->bootinfo);
}
...
```

- QEMU 실행할 때 -smp 옵션으로 주어진 CPU의 개수만큼 CPU 추가
- QEMU에서 가상화나 TrustZone Emulation 사용 안 함
- CBAR라는 CPU 시스템 레지스터에 GIC 주소 등록
- GIC는 바로 다음페이지에서 다룹니다.
- PSCI의 방법을 HVC로 지정
- 반드시 arm_load_kernel은 마지막 위치

GIC (Generic Interrupt Controller) 란?

인터럽트 컨트롤러 (INTC, Interrupt controller) 란?

- CPU와 많은 디바이스 사이를 중재하여 인터럽트를 중간에서 분배하고 제어하는 역할
 - CPU가 많은 디바이스와 연결될 만큼 물리적인 핀(Pin)이 많지 않기 때문
 - 인터럽트가 동시에 들어올 경우 어떤 인터럽트가 실행될지 우선순위를 정함

공통된 인터럽트 처리 방식을 제공하기 위해 ARM에서 GIC를 도입

- ARM에서 GIC를 제공하기 전에는 SoC마다 각각 인터럽트 컨트롤러가 달랐음
- ARM CPU에 가상화 관련 기능이 들어오면서 인터럽트 처리가 복잡해짐

꾸준히 개발되어 왔으며 현재는 버전 4까지 출시됨

➤ 본 수업에서는 가장 보편적으로 사용되는 버전 3를 사용합니다.

인터럽트의 종류

GIC에서의 인터럽트 종류

- SPI (Shared Peripheral Interrupt) : 모든 CPU가 공유하는 인터럽트, 32~1019 번 ID 사용
 - CPU 외부의 디바이스가 발생시킴
- PPI (Private Peripheral Interrupt) : 특정 CPU만 사용하는 인터럽트, 16~31번 ID 사용
 - 일반적으로 CPU 내의 디바이스 (예> 타이머)가 발생시킴
- SGI (Software Generated Interrupt) : GIC 레지스터로 세팅으로 발생하는 인터럽트, 0~15번 ID 사용
 - CPU 간에 신호를 보내기 위한 용도로 사용

ARM CPU에서의 인터럽트 종류

- IRQ : 일반적인 인터럽트에 사용
 - FIQ : 일반적인 인터럽트보다 우선순위가 높게 처리해야 하는 경우 사용
- 리눅스에서는 일반적으로 FIQ를 사용하지 않고 IRQ만으로 인터럽트를 처리합니다.

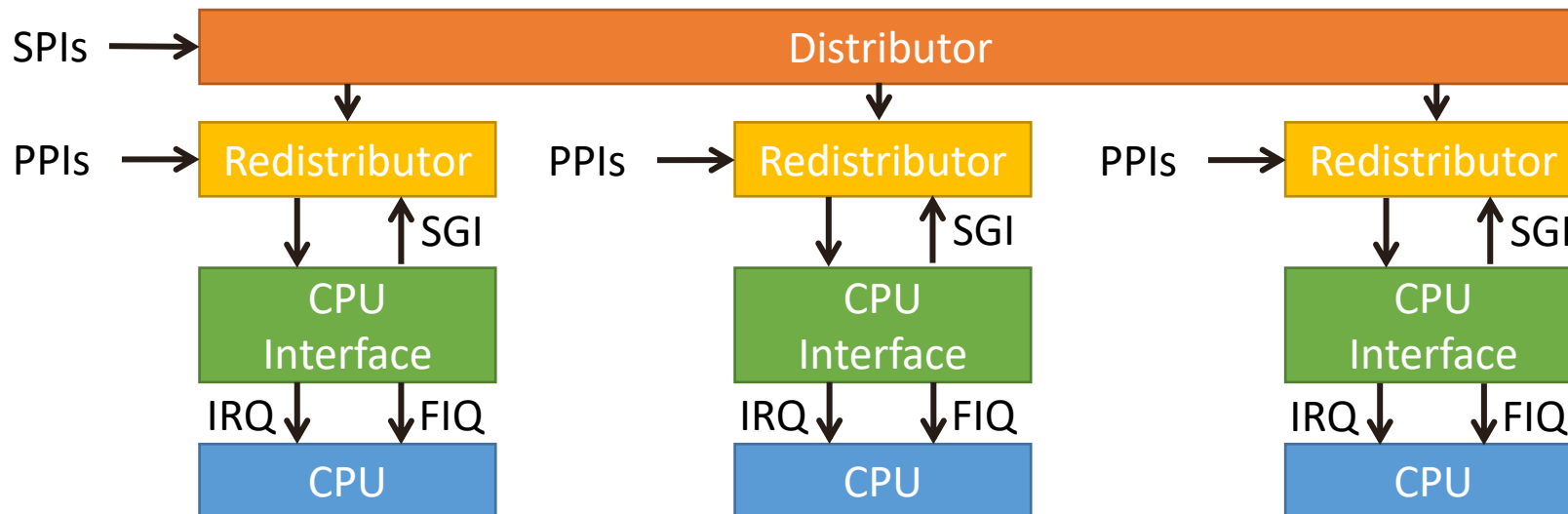
GIC의 구조

디스트리뷰터 (Distributor) : CPU 외부 디바이스들로부터 SPI를 받아 제어하는 GIC 모듈

- 각각의 SPI 인터럽트의 우선순위를 설정, 활성화/비활성화, 인터럽트 방식 등을 설정

리디스트리뷰터 (Redistributor) : PPI와 SGI를 받아 제어하고 SPI를 중개하는 GIC 모듈

- 각각의 SGI, PPI 인터럽트의 우선순위를 설정, 활성화/비활성화, 인터럽트 방식 등을 설정



CPU 타이머

타이머 : 시분할 스케줄링을 구현하기 위해 가장 필수가 되는 디바이스

- 일정 시간 간격으로 인터럽트를 주기적으로 발생시킴
- 타이머 디바이스가 없으면 리눅스가 부팅되지 않음

ARM CPU에서 기본적으로 제공되는 타이머가 존재

- 기본적으로는 커널 모드(EL1)가 사용하기 위한 타이머 디바이스 존재
- 하이퍼바이저나 TrustZone에서 스케줄링을 위한 타이머 디바이스들도 추가로 존재함

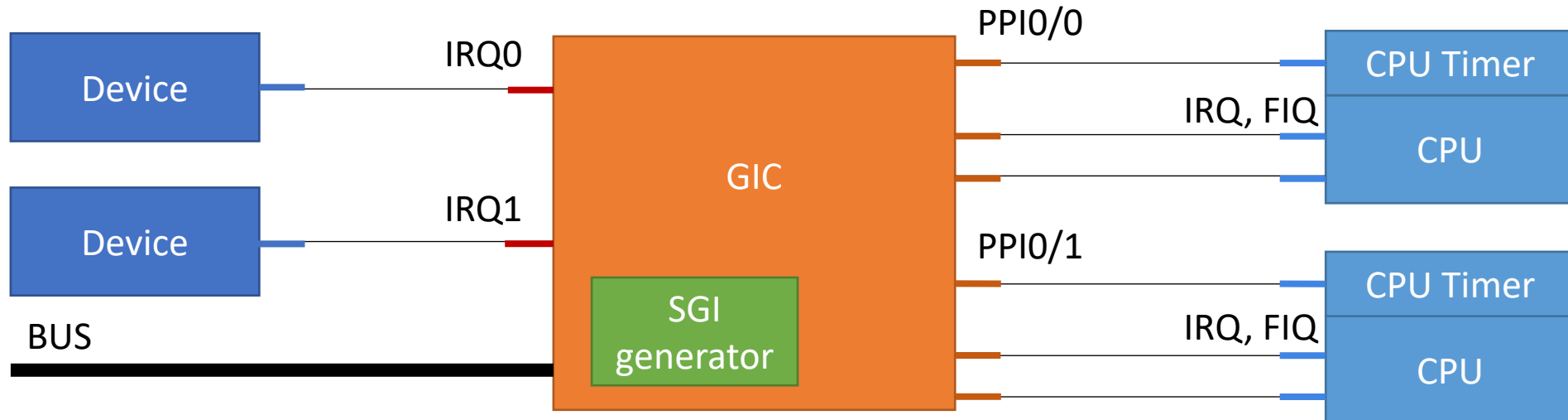
➤ 본 수업에서는 커널모드용 타이머 디바이스만 사용하겠습니다.

GIC의 하드웨어적인 구조

인터럽트는 물리적인 핀에 전압이 감지가 되면 발생함

- 전압이 감지된 상태를 보통 High, 전압이 없는 상태를 Low라고 함
- 물리적인 핀이 계속 High 상태라면 인터럽트는 계속 발생함
 - 인터럽트를 끄기 위해서는, 해당 디바이스에 대한 처리를 완료해서, 디바이스가 Low로 상태를 바꾸어야 함

각 핀들은 전압 신호를 주기 위해 물리적인 전선으로 연결되어 있음



QEMU의 새 SoC에 GIC 추가하기 (1/2)

```
#include "hw/intc/arm_gicv3_common.h"
...
#define NUM_IRQS 256

#define ARCH_TIMER_NS_EL1_IRQ 14

#define COMENTO_BASE_GIC_REDIST 0x08000000
#define COMENTO_BASE_GIC_DIST 0x09000000
#define COMENTO_SIZE_GIC_REDIST 0x01000000
...
struct ComentoMachineState {
...
    DeviceState *gic;
};

static void create_gic(ComentoMachineState *vms)
{
    MachineState *ms = MACHINE(vms);
    SysBusDevice *gicbusdev;
    unsigned int smp_cpus = ms->smp_cpus;

    vms->gic = qdev_new("arm-gicv3");

    qdev_prop_set_uint32(vms->gic, "revision", 3);
    qdev_prop_set_uint32(vms->gic, "num-cpu", smp_cpus);
    qdev_prop_set_uint32(vms->gic, "num-irq", NUM_IRQS + 32);

    uint32_t redist_capacity = COMENTO_SIZE_GIC_REDIST / GICV3_REDIST_SIZE;
    uint32_t redist_count = MIN(smp_cpus, redist_capacity);
    qdev_prop_set_uint32(vms->gic, "len-redist-region-count", 1);
    qdev_prop_set_uint32(vms->gic, "redist-region-count[0]", redist_count);
}
```

• GIC의 메모리 주소 정의

• 다른 디바이스가 인터럽트를 사용하기 위해서는 GIC와 연결 필요

- GIC를 버전3으로 생성
- GIC에 IRQ 개수와 CPU 개수를 설정
- IRQ 개수에 PPI + SGI 개수인 32를 더 함
- 리디스트리뷰터 개수 설정
- 영역 크기는 0x100_0000이므로 엔트리 크기 0x2_0000로 나누어주면 128개

QEMU의 새 SoC에 GIC 추가하기 (2/2)

```
gicbusdev = SYS_BUS_DEVICE(vms->gic);
sysbus_realize_and_unref(gicbusdev, &error_fatal);
sysbus_mmio_map(gicbusdev, 0, COMENTO_BASE_GIC_DIST);
sysbus_mmio_map(gicbusdev, 1, COMENTO_BASE_GIC_REDIST);

int i;
for (i = 0; i < smp_cpus; i++) {
    DeviceState *cpudev = DEVICE(qemu_get_cpu(i));
    int ppibase = NUM_IRQS + i * GIC_INTERNAL + GIC_NR_SGIS;

    qdev_connect_gpio_out(cpudev, 0,
        qdev_get_gpio_in(vms->gic, ppibase + ARCH_TIMER_NS_EL1_IRQ));

    sysbus_connect_irq(gicbusdev, i,
        qdev_get_gpio_in(cpudev, ARM_CPU_IRQ));
    sysbus_connect_irq(gicbusdev, i + smp_cpus,
        qdev_get_gpio_in(cpudev, ARM_CPU_FIQ));
}
...
static void machcomento_init(MachineState *machine)
{
    ...
    create_gic(vms);
}
```

- GIC를 메모리에 매핑
- GIC에서 사용되는 핀 중 현재 CPU의 PPI의 시작 번호 계산
- 현재 CPU에서 사용하는 Timer를 GIC의 핀에 연결함
- CPU의 IRQ와 FIQ핀에 GIC를 연결

Chapter. 4

디바이스 트리 파악하기

디바이스 트리 (Device Tree) 란?

현재 SoC의 디바이스에 대한 정보를 트리 형태로 적어 놓은 데이터

- .dts라는 확장자인 스크립트를 작성하고, 빌드하면 .dtb라는 형태의 바이너리로 나옴

디바이스 트리가 제공되기 전에는 SoC 각각마다 리눅스 코드가 각각 작성되었음

- 같은 디바이스를 사용하더라도 제조사의 사정에 따라 메모리 주소나 인터럽트 번호가 달랐기 때문
 - 기존에는 헤더파일 내에 매크로 등의 형태로 메모리 주소와 인터럽트 번호가 하드코딩 됨
 - ARM SoC가 폭발적으로 증가하면서 기존 방법으로는 중복되는 코드가 너무 많아짐

QEMU에서 virt 머신을 사용하여 부팅할 때는 디바이스 트리가 없었는데?

- QEMU의 virt 머신은 자동으로 디바이스 트리 바이너리를 만들어 줌
- 하지만, 실제 SoC의 경우는 리눅스 빌드시 디바이스 트리도 같이 빌드해야 함

➤ 본 수업에서는 실제 SoC 처럼 리눅스에서 디바이스 트리를 작성해 보겠습니다.

커널에 새로운 디바이스 트리 추가하기

```
config ARCH_COMENTO
    bool "Comento SoC"
    help
        This enables support for
        Comento Soc family
```

1. 새로운 config를 아래와 같이 Kconfig.platform 마지막에 추가 `<arch/arm64/Kconfig.platform>`
2. arch/arm64/boot에 새로운 디렉토리를 만들고, Makefile에 아래와 같이 디렉토리 추가
`subdir-y += comento` `<arch/arm64/boot/dts/Makefile>`
3. 추가한 디렉토리에 Makefile을 아래와 같이 추가
`dtb-$(CONFIG_ARCH_COMENTO) += comento.dtb` `<arch/arm64/boot/comento/dts/Makefile>`
4. 커널 빌드할 때 추가했던 defconfig에 추가한 config를 아래와 같이 추가
`CONFIG_ARCH_COMENTO=y` `<arch/arm64/configs/comento_defconfig>`
5. 수정한 defconfig를 make 명령어를 사용하여 적용
 - ARCH=arm64 make comento_defconfig

새로운 디바이스 트리 스크립트 작성하기

디바이스 트리는 버전을 나타내는 `/dts-v1/`으로 시작함

디바이스 트리는 `/`(루트 노드)가 존재하여야 함, 모든 노드는 루트 노드 내에 속해야 함

- 노드는 `{`와 `}` 사이에 자신의 속성들을 명시
 - 속성은 `"속성이름" = "속성 값";` 또는 값 없이 `"속성이름";`으로 명시 가능
 - 속성 값에는 `"<"`와 `">"`로 묶여있는 셀(Cell) 또는 문자열이 올 수 있음
 - 속성 이름 앞에 `"#"`이 붙은 경우는 개수를 나타내는 속성임을 의미함

루트 노드는 아래와 같은 속성을 반드시 가져야 함

```
/dts-v1/;

/ {
    model = "Comento SoC";
    compatible = "comento,comento-v1";
    interrupt-parent = <&gic>;
    #size-cells = <2>;
    #address-cells = <2>;
};
```

- SoC 모델 종류
- 리눅스에서 노드를 검색할 때 사용할 문자열
- 현재 SoC에서 사용되는 인터럽트 컨트롤러
- Size를 나타내는 cell의 사이즈는 반드시 2여야 함을 명시
- 메모리 주소를 나타내는 cell의 사이즈는 반드시 2여야 함을 명시

`<arch/arm64/boot/dts/coment/comento.dts>`

QEMU의 새 SoC에 메모리와 CPU와 PSCI 노드 추가하기

```
memory@40000000 {
    reg = <0x00 0x40000000 0x00 0x1000000>;
    device_type = "memory";
};
cpus {
    #size-cells = <0>;
    #address-cells = <1>;

    cpu@0 {
        reg = <0>;
        enable-method = "psci";
        compatible = "arm,cortex-a72";
        device_type = "cpu";
    };

    cpu@1 {
        reg = <1>;
        enable-method = "psci";
        compatible = "arm,cortex-a72";
        device_type = "cpu";
    };
};
psci {
    migrate = <0xc4000005>;
    cpu_on = <0xc4000003>;
    cpu_off = <0x84000002>;
    cpu_suspend = <0xc4000001>;
    method = "hvc";
    compatible = "arm,psci-1.0", "arm,psci-0.2", "arm,psci";
};
```

- 메모리의 시작 주소와 크기 지정
- CPU 지정시 사용되는 셀 크기 지정
- CPU에서 크기는 무의미하므로 0
- CPU 개수가 32비트 하나로 표현 가능하므로 0
- CPU 2개를 지정
- CPU를 활성화하기 위해서는 PSCI 사용
- CPU의 종류는 Cortex-a72
- Reg에는 CPU의 번호를 명시함
- 노드 이름에서 @ 뒤는 구분을 위해, reg의 값을 명시해줌
- PSCI 방법을 명시
- migrate ~ cpu_suspend는 하이퍼콜 번호
- 방법은 hvc, arm-psci 0.2/1.0과 호환

QEMU의 새 SoC에 GIC와 Timer 노드 추가하기

```
#include <dt-bindings/interrupt-controller/arm-gic.h>
...
gic: intc@8000000 {
    compatible = "arm,gic-v3";
    reg = <0x00 0x90000000 0x00 0x10000>,
        <0x00 0x80000000 0x00 0x1000000>;
    ranges;
    interrupt-controller;
    #redistributor-regions = <1>;
    #size-cells = <2>;
    #address-cells = <2>;
    #interrupt-cells = <3>;
};

timer {
    interrupts = <GIC_PPI 0 IRQ_TYPE_LEVEL_HIGH>,
                <GIC_PPI 14 IRQ_TYPE_LEVEL_HIGH>;
    always-on;
    compatible = "arm,armv8-timer", "arm,armv7-timer";
};
```

- 헤더파일은 /dts-v1/; 바로 뒤에
- GIC 명시
 - Reg에는 메모리 주소와 크기를 명시
 - 인터럽트를 명시 할때는 3개의 integer를 사용하도록 #interrupt-cells로 명시
 1. 인터럽트의 종류
 2. 인터럽트 ID
 3. 인터럽트 활성화 일때 Pin 상태 (High/Low)
- CPU 타이머 명시
- Interrupt는 Secure, Non-secure, Virtual, Hypervisor 순으로 명시해야 함
 - Trustzone이나 가상화는 사용하지 않으므로, Non-secure만 명시함
 - Secure는 0으로 하고 나머지는 명시하지 않음

디바이스 트리 문법

디바이스 트리의 루트노드에 대한 기본적인 문법은 devicetree 사이트에서 정의

- <https://www.devicetree.org/specifications/>
 - 루트 노드 : <https://devicetree-specification.readthedocs.io/en/latest/chapter3-devicenodes.html>

리눅스 커널이 지원하는 디바이스는 Documentation/devicetree 에 정의 되어 있음

- 디바이스 드라이버 마다 자신만의 다양한 문법을 정의함
 - 디바이스 트리 문법에 헛갈리는 부분이 있을 때는 문서를 참고하면 좋음
- <https://elixir.bootlin.com/linux/latest/source/Documentation/devicetree/bindings>
 - GIC : <https://elixir.bootlin.com/linux/v6.7/source/Documentation/devicetree/bindings/interrupt-controller/arm,gic-v3.yaml>
 - 타이머 : https://elixir.bootlin.com/linux/v6.7/source/Documentation/devicetree/bindings/timer/arm,arch_timer.yaml



질문해주세요!

이해되지 않거나, 궁금한 내용은 채팅과 마이크를 통해 질문해주세요.
질문이 많을수록 더 많은 걸 알려드릴 수 있어요.



쉬는 시간!

조금 쉬었다가, 다시 만나요!

Chapter. 5

새 SoC에서 리눅스 부팅하기

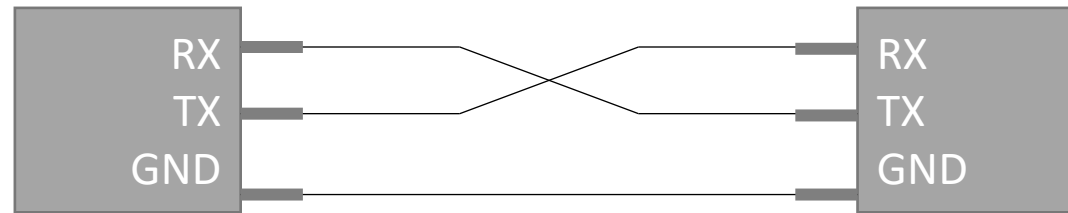
UART (Universal Asynchronous Receiver/Transmitter) 란?

일반적으로 시리얼 통신이라고 부르며, 송신/수신/그라운드의 3개 선으로 1:1 통신 가능

- 다양한 데이터를 주고 받을 수 있으나 일반적으로는 터미널을 위해 주로 사용

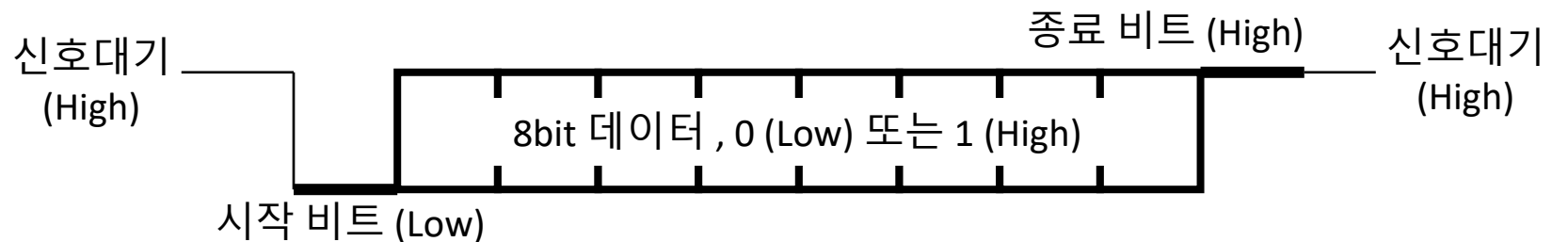
UART 하드웨어 구조

- RX와 TX가 서로 반대로 연결



UART 데이터 전송 방식

- High에서 Low로 바뀌면 신호 읽기 시작



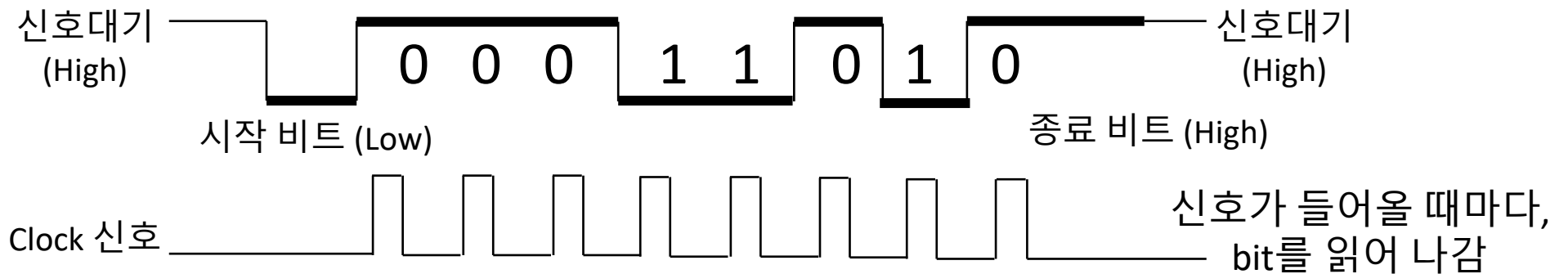
➤ 본 수업에서는 SoC에서 부팅한 리눅스에서 명령어를 사용하기 위해 UART를 이용합니다.

Clock 이란?

타이밍을 유지하기 위한 신호를 만드는 장치

UART에서 각각의 bit를 처리하기 위한 타이밍은? 1이 연속으로 있다면? 0이 연속으로 있다면?

- 송신 측과 수신 측이 동일한 타이밍을 유지하기 위한 신호가 필요



UART에서는 clock의 속도인 baudrate 값을 공유하고 각자의 clock으로 신호를 만들

- clock의 타이밍 신호를 직접 공유한다면? USART (선 하나가 추가로 필요)

QEMU의 새 SoC에 Uart 추가하기

```
#include "hw/char/pl011.h"
...
#define COMENTO_IRQ_UART      0
...
#define COMENTO_BASE_UART     0x09010000
...
static void create_uart(const ComentoMachineState *vms, MemoryRegion *mem)
{
    hwaddr base = COMENTO_BASE_UART;
    int irq = COMENTO_IRQ_UART;
    DeviceState *dev = qdev_new(TYPE_PL011);
    SysBusDevice *s = SYS_BUS_DEVICE(dev);

    qdev_prop_set_chr(dev, "chardev", serial_hd(0));
    sysbus_realize_and_unref(s, &error_fatal);

    memory_region_add_subregion(mem, base, sysbus_mmio_get_region(s, 0));
    sysbus_connect_irq(s, 0, qdev_get_gpio_in(vms->gic, irq));
}
...
static void machcomento_init(MachineState *machine)
{
    ...
    create_uart(vms, sysmem);
}
```

- ARM PL011 UART 장치 추가
- 옵션으로 지정된 QEMU 외부 입출력
- 추가한 QEMU내 UART의 입출력이 연결됨
- 옵션 지정 없으면, serial_hd(0)은 표준 입출력
- UART를 메모리와 GIC에 매핑

디바이스 트리에 UART, clock 추가하기

```
clock: apb-pclk {
    clock-output-names = "clk24mhz";
    clock-frequency = <24000000>;
    compatible = "fixed-clock";
    #clock-cells = <0>;
};

pl011@9010000 {
    compatible = "arm,pl011", "arm,primecell";
    clock-names = "uartclk", "apb_pclk";
    clocks = <&clock>, <&clock>;
    interrupts = <GIC_SPI 0 IRQ_TYPE_LEVEL_HIGH>;
    reg = <0x00 0x9010000 0x00 0x1000>;
};
```

- QEMU 에뮬레이션의 한계로 실제 clock 장치가 존재하지는 않음
- 리눅스 내에서 각종 디바이스 초기화를 위해 Mock clock 디바이스를 명시
- UART 디바이스 정보 명시
- UART 디바이스가 사용할 clock 명시
- UART 입력시 발생하는 인터럽트 명시
- UART의 메모리 주소와 크기 명시

Initramfs 란?

초기화 과정에서 사용되는 읽기 전용의 소형의 루트파일시스템

- 부팅 이후 파일 수정은 가능하나 수정사항이 반영되지는 않음

부트로더 등이 리눅스가 실행되기 전에 메모리에 올려줌

- QEMU에서는 -initrd라는 옵션을 사용하여 지정 가능

CPIO라는 형식의 묶은 파일로 되어있으며 gzip 등으로 압축이 되기도 함

Initramfs를 사용하기 위해서는 커널 설정 추가 필요

```
CONFIG_BLK_DEV_INITRD=y  
CONFIG_RD_GZIP=y
```

<arch/arm64/configs/comento_defconfig>

- 수정한 defconfig를 make 명령어를 사용하여 적용 : ARCH=arm64 make comento_defconfig

- 리눅스의 부팅이 완료되기 위해서는 루트파일시스템이 필요합니다.
- 아직 저장장치가 SoC에 없으므로, initramfs를 사용하도록 하겠습니다.

새 SoC에서 리눅스 실행하기

<QEMU 디렉토리/build/qemu-system-aarch64 ₩
-kernel <리눅스 디렉토리>/arch/arm64/boot/Image ₩
-initrd <빌드루트 디렉토리>/output/images/rootfs.cpio.gz ₩
-append "console=ttyAMA0" ₩
-dtb <리눅스 디렉토리>/arch/arm64/boot/dts/comento/comento.dtb ₩
-nographic -M comento -m 1G -smp 2

```
Run /init as init process
Starting syslogd: OK
Starting klogd: OK
Running sysctl: OK
Saving 256 bits of non-creditable seed for next boot
Starting network: Waiting for interface eth0 to appear..... timeout!
run-parts: /etc/network/if-pre-up.d/wait_iface: exit status 1
FAIL

Welcome to Buildroot
buildroot login: █
```

실행된 리눅스에서 메모리맵 확인하기

cat /proc/iomem

```
# cat /proc/iomem
08000000-08ffffff : GICR
09000000-0900ffff : GICD
09010000-09010fff : pl011@9010000
    09010000-09010fff : 9010000.pl011 pl011@9010000
40000000-7fffffff : System RAM
    40210000-40afffff : Kernel code
    40b00000-40b9ffff : reserved
    40ba0000-40ceffff : Kernel data
    48000000-48445fff : reserved
    48600000-48605fff : reserved
    7ab40000-7fbfffff : reserved
    7fc3f000-7fdbffff : reserved
    7fdc0000-7fde3fff : reserved
    7fde6000-7fde6fff : reserved
    7fde7000-7fde7fff : reserved
    7fde8000-7fdfafff : reserved
    7fdfb000-7fffffff : reserved
```



질문해주세요!

이해되지 않거나, 궁금한 내용은 채팅과 마이크를 통해 질문해주세요.
질문이 많을수록 더 많은 걸 알려드릴 수 있어요.

과제 안내

과제 의도

- 주어진 기준에 맞춰서 새로운 SoC를 QEMU에 구성해 본다.
- 현업에서 자주 사용 되는 디바이스 트리의 사용법을 익힐 수 있다.

필수 과제 내용

1. 본인의 영어 ini설로 된 machine을 QEMU에 구성하고 디바이스 트리도 작성한다.
 1. RAM 시작주소는 0x8000_0000, 크기는 1GB
 2. GIC의 디스트리뷰터 시작주소는 0x0400_0000, 리디스트리뷰터 시작주소는 0x0300_000, 크기는 16MB
 3. UART의 시작주소는 0x0401_0000
2. 추가한 QEMU 소스코드 파일과 리눅스의 디바이스 트리 업로드하여 제출한다.

과제 안내

선택 과제 내용

1. UART를 하나 더 추가(시작 주소 0x0402_0000)하고 테스트 해본다.
 1. QEMU의 -serial 옵션을 통해 serial_hd(0)와 serial_hd(1)을 순차적으로 지정할 수 있다.
 - 옵션 사용 예시 : -serial mon:stdio -serial tcp::45456,server,nowait
 - netcat localhost <포트번호>를 사용하면 tcp 소켓으로 출력되는 uart 출력을 화면에 표시할 수 있다.
 2. /dev/ttyAMA0, /dev/ttyAMA1과 같이 순차적으로 디바이스가 생기게 된다.
 - echo나 cat 명령어로 새로 만들어진 디바이스에 텍스트를 보내거나 읽어보자
2. 추가한 QEMU 소스코드 파일과 리눅스의 디바이스 트리 업로드하여 제출한다.
3. UART가 잘 동작함을 테스트하는 화면을 스크린샷 캡처하고 업로드하여 제출한다.

기한 : 다음 주 월요일 오전 0시까지

다음 주 커리큘럼

- AMBA 버스 이해하기
- MMIO 개념 파악하기
- 인터럽트 개념 이해하기
- 새로운 MMIO 하드웨어 만들기
- MMIO 하드웨어 드라이버 만들기

